



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Microprocesadores y Microcontroladores.

Práctica No. 2: Programación en Lenguaje Ensamblador del ATmega1280/2560.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Jesús A. García L.

Agosto/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	3
	Teoría	3
	Registros de propósito general y el archivo de definiciones (.inc).....	3
	Instrucciones usadas en el programa	4
	Código fuente (Prac2.asm)	4
	¿Qué hace el programa?	4
	Resultados de simulación	4
	Reflexión y análisis crítico	5
	Dificultades técnicas.....	5
	Fracasos, fallas, bugs y errores.....	5
	Cómo se resolvió.....	5
	Conclusión	5
IV.	Bibliografía.....	5

I. Objetivo

El alumno usará AVR Studio como herramienta de programación en lenguaje ensamblador para el sistema ATmega1280/2560: creará un proyecto de ensamblador, escribirá y compilará un programa fuente (.asm), y lo simulará paso a paso para observar el comportamiento de los registros del CPU.

II. Descripción de la practica

Instalar AVR Studio 5 (IDE de Atmel/Microchip) en la computadora personal.

Crear un archivo de texto con extensión .asm con el código del Listado 1.

Crear en la raíz (C:\) una carpeta llamada uC y, dentro de esta, otra llamada Prac2, quedando la ruta C:\uC\Prac2.

Crear en AVR Studio un proyecto de ensamblador llamado Prac2 en esa ruta, con la opción "Create directory for solution" activada.

Seleccionar el dispositivo (Device) ATmega2560.

Agregar el archivo fuente (Prac2.asm) al proyecto, en la carpeta Source Files, y escribir en él el código del Listado 1.

Compilar el proyecto (F7) y corregir cualquier error o warning antes de continuar.

Abrir el simulador y ejecutar el programa paso a paso, observando el cambio en el registro R24.

Determinar qué hace el programa, redactar las conclusiones y comentarios de la práctica, y subir el código (Prac2.asm) a un repositorio junto con el reporte.

Lista de materiales

- Computadora personal (PC).
- AVR Studio 5 (IDE de Atmel/Microchip).
- Editor de texto y terminal con acceso a git.
- Datasheet del ATmega2560 / AVR Instruction Set Manual (para las instrucciones usadas en el Listado 1).

III. Desarrollo de la práctica

Teoría

Registros de propósito general y el archivo de definiciones (.inc)

El ATmega1280/2560 tiene 32 registros de propósito general (R0-R31) de 8 bits; el programa usa R24 como contador. AVR Studio liga cada proyecto a un dispositivo específico (elegido al crearlo, en este caso ATmega2560) y, con base en ese dispositivo, incluye automáticamente sus definiciones de registros e I/O. El archivo .inc (por ejemplo m2560def.inc) referenciado con `.INCLUDE` en el código fuente debe corresponder al mismo dispositivo seleccionado en el proyecto; si no coincide, sus definiciones (por ejemplo los alias XH/XL, YH/YL, ZH/ZL) se cargan dos veces y el ensamblador marca warnings de redefinición.

Instrucciones usadas en el programa

El Listado 1 usa solo seis instrucciones: LDI (carga un valor constante en un registro, solo para R16-R31), INC y DEC (incrementan/decrementan un registro en 1), CPI (compara un registro contra una constante, sin modificarlo, y actualiza banderas), BREQ (salta a una etiqueta si la comparación anterior dio igualdad, es decir, si la bandera Z quedó en 1) y RJMP (salto relativo incondicional a una etiqueta).

Código fuente (Prac2.asm)

El proyecto se creó para el dispositivo ATmega2560 y el archivo fuente quedó como sigue:

```
/*
 * Prac2.asm
 *
 * Created: 04/09/2026 09:17:57 p. m.
 * Author: Okuyt
 */
;----- definiciones e includes -----
.INCLUDE "m2560def.inc" ; Incluir definiciones de Registros para 1280
.equ INIT_VALUE = 0 ; Valor inicial R24
;----- inicializar -----
ldi R24,INIT_VALUE
;----- ciclo principal -----
arriba: inc R24
cpi R24,10
breq abajo
rjmp arriba
abajo: dec R24
cpi R24,0
breq arriba
rjmp abajo
;-----
```

¿Qué hace el programa?

R24 se inicializa en 0. En la etiqueta arriba, el programa incrementa R24 (INC) y compara contra 10 (CPI); mientras no sea igual, RJMP regresa a arriba, así que R24 sube 0,1,2,...,10. Al llegar a 10, BREQ salta a abajo, donde el programa decrementa R24 (DEC) y compara contra 0; mientras no sea igual, RJMP regresa a abajo, así que R24 baja 10,9,8,...,0. Al llegar a 0, BREQ salta de vuelta a arriba y el ciclo se repite indefinidamente: R24 oscila como un contador triangular entre 0 y 10 (0→1→...→10→9→...→0→1→...), sin detenerse nunca, ya que no hay ninguna instrucción de alto.

Resultados de simulación

Ensamblando y ejecutando el programa paso a paso en el simulador de AVR Studio, con breakpoints en las etiquetas arriba y abajo, se observó en la ventana de registros el siguiente comportamiento de R24:

```
R24 = 0      (ldi, valor inicial)
R24 = 1..9   (arriba: inc, cpi, rjmp arriba)
R24 = 10     (arriba: inc -> cpi R24,10 -> breq abajo)
R24 = 9..1   (abajo: dec, cpi, rjmp abajo)
```

```
R24 = 0      (abajo: dec -> cpi R24,0 -> breq arriba)
... (el ciclo arriba/abajo se repite indefinidamente)
```

Reflexión y análisis crítico

Dificultades técnicas

La principal dificultad fue un warning de redefinición al compilar (Preprocessor: Redefinition of macro '__PART_NAME__'; .def: 'XH'/'XL'/'YH'/'YL'/'ZH'/'ZL' redefinition; Redefinition of macro '__FLASH_SIZE__'), que no impedía compilar pero indicaba una inconsistencia en la configuración del proyecto.

Fracasos, fallas, bugs y errores

El origen del problema fue que el proyecto en AVR Studio estaba configurado para el dispositivo ATmega2560, pero en un intento previo el archivo fuente incluía las definiciones de otro dispositivo (m1280def.inc); AVR Studio ya inserta automáticamente las definiciones del dispositivo seleccionado en el proyecto, así que al incluir además, a mano, el archivo .inc de un dispositivo distinto, los mismos registros (XH, XL, YH, YL, ZH, ZL) y macros (__PART_NAME__, __FLASH_SIZE__) quedaban definidos dos veces.

Cómo se resolvió

Se corrigió la línea .INCLUDE del archivo fuente para que apuntara al archivo de definiciones del dispositivo realmente seleccionado en el proyecto (m2560def.inc en vez de m1280def.inc), con lo que las definiciones dejaron de duplicarse y los warnings desaparecieron.

Conclusión

Esta práctica permitió usar AVR Studio de principio a fin: crear un proyecto de ensamblador ligado a un dispositivo específico, escribir un programa fuente sencillo, resolver un problema real de configuración (archivo de definiciones .inc que no correspondía al dispositivo del proyecto) y simular el programa paso a paso para verificar, registro por registro, que efectivamente implementa un contador triangular (cuenta ascendente de 0 a 10 y descendente de 10 a 0, en bucle infinito) sobre R24.

IV. Bibliografía

- [1] Microchip Technology / Atmel. ATmega640/V-1280/V-1281/V-2560/V-2561/V Datasheet.
- [2] Atmel Corporation. AVR Studio 5 User Guide.
- [3] Universidad Autónoma de Baja California, Facultad de Ciencias Químicas e Ingeniería. (2026). Microprocesadores y Microcontroladores — Práctica 2: Programación en Lenguaje Ensamblador del ATmega1280 (hoja de práctica, S25-1).